

Блок	Чек-лист	Место для заполнения
	Зачем автору реализация данного курса? Какова общая задача курса? Какие образовательные цели программы закрывает данный курс?	Сергей Васильев - самоучка, разрабатывает с 2018 года, который ценит бесплатные и доступные знания. Для него это способ структурировать собственный опыт, поделиться практикой ("приятное хобби") и помочь другим пройти путь, который он освоил самостоятельно. Он верит, что для успеха достаточно "желания и наличия свободного времени". Общая задача: Дать слушателям быстрый и практический инструмент для создания веб-приложений, минуя длительную теоретическую подготовку, характерную для классического образования. Образовательные цели: Закрытие разрыва между теоретическим знанием Python и реальными задачами веб-разработки (API, бэкенд). Формирование навыка работы с современным стеком (FastAPI вместо устаревающих решений).

	Каковы будут последствия если слушатели не пройдут данный курс?	Студенты могут потратить значительно больше времени на поиск актуальной информации (как это делал автор в начале пути). Есть риск освоения устаревших подходов (например, только синхронные фреймворки вроде Flask/Django там, где эффективнее FastAPI). Отсутствие структурированного пути может снизить мотивацию из-за сложности самостоятельной настройки окружения и понимания архитектуры API.
	Какие конкретно навыки/знания должны появиться в руках студентов после завершения данного курса?	Умение разворачивать проект на FastAPI, писать маршруты, использовать Pydantic для валидации, подключать базы данных, настраивать аутентификацию, умение работать с Git, Docker, Linux/bash.

Интервью с автором	Какие темы, касающиеся данного курса, студенты уже должны освоить в рамках других дисциплин/курсов?	Базовый Python: Синтаксис, типы данных, функции, основы ООП (классы, наследование). Основы веба: Понимание клиент-серверной архитектуры, что такое запрос и ответ. Базы данных: Принципы реляционных БД, базовый SQL.
	Какие темы необходимо поднять в рамках курса, чтобы студенты могли перейти к изучению других курсов, связанных с данным?	Асинхронное программирование: Глубокое понимание async/await (критично для FastAPI). Микросервисная архитектура: Как масштабировать приложение. DevOps и CI/CD: Автоматизация деплоя, мониторинг. Безопасность: Углубленная защита данных.

	Оцените сложность обучения и подчеркните основные "подводные камни", с которыми могут столкнуться обучающиеся	Сложность: Средняя. Курс требует самостоятельности (формат Stepik + текст). Подводные камни: Настройка окружения: Docker может вызывать трудности у новичков. Асинхронность: Концепция async может быть сложна для тех, кто писал только синхронный код. Самодисциплина: Отсутствие жестких дедлайнов и вебинаров требует высокой внутренней мотивации (автор делает ставку на "желание и время").
	Каковы ожидания от результатов обучающихся?	Студент способен самостоятельно создать бэкенд для простого сервиса (например, бота или сайта). Понимает, как искать решения проблем (опыт автора: YouTube, Stepik, телеграм-каналы). Имеет портфолио-проект (сертификат + код на GitHub). Готов к выполнению простых фриланс-задач или стажировке.

	Какие сложности могут возникнуть при работе с данной группой (целевой аудиторией)?	Разный бэкграунд: В группе могут быть как полные новички, так и опытные разработчики. Технические барьеры: Проблемы с установкой ПО на личных компьютерах студентов. Отсутствие синхронной связи: Вопросы решаются через комментарии, что может быть медленно для тех, кто привык к очным консультациям.
--	--	--